

Lecture 6: Using a Neural Network to Fit the Data (2) & Exam Review

Recap: The Learning Process

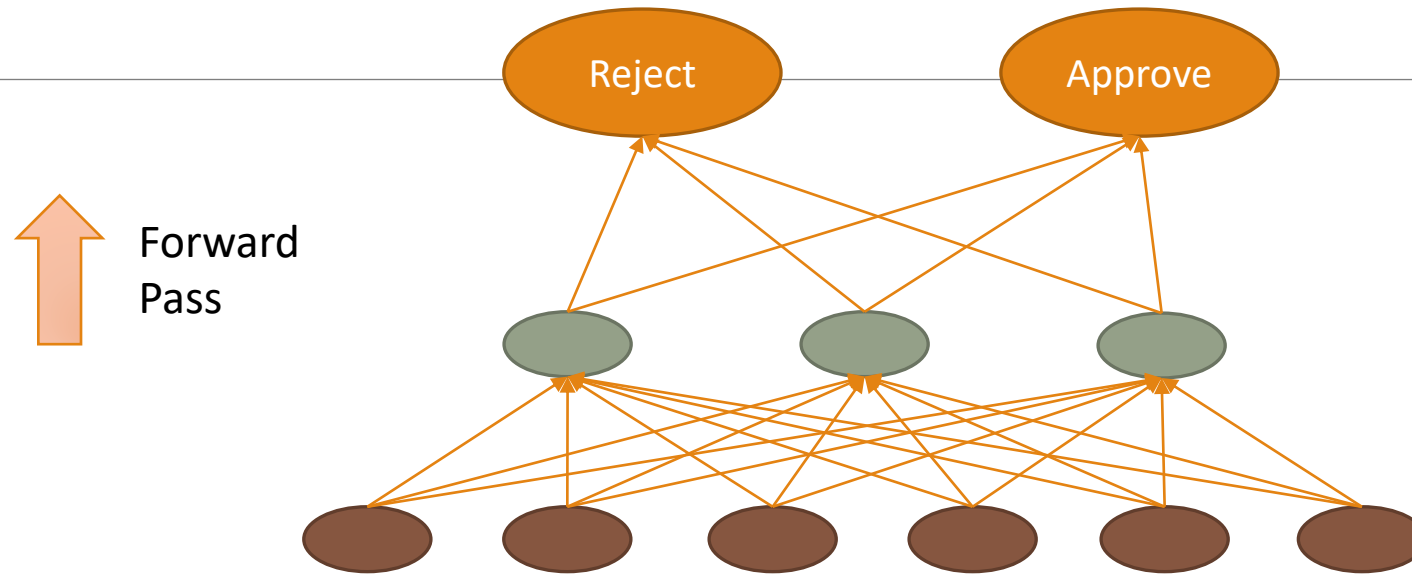
- **Forward pass**

- Given input data and initial values for the weights, the model is fed input data

- **Backward pass**

- A measure of the error is evaluated by comparing the outputs to the ground truth
 - The gradient of the error with respect to the parameters is computed using the chain rule
 - Optimize the parameter of the model: the value of the weights is then updated in the direction that leads to a decrease in the error.
- The procedure is repeated until the error falls below an acceptable level.

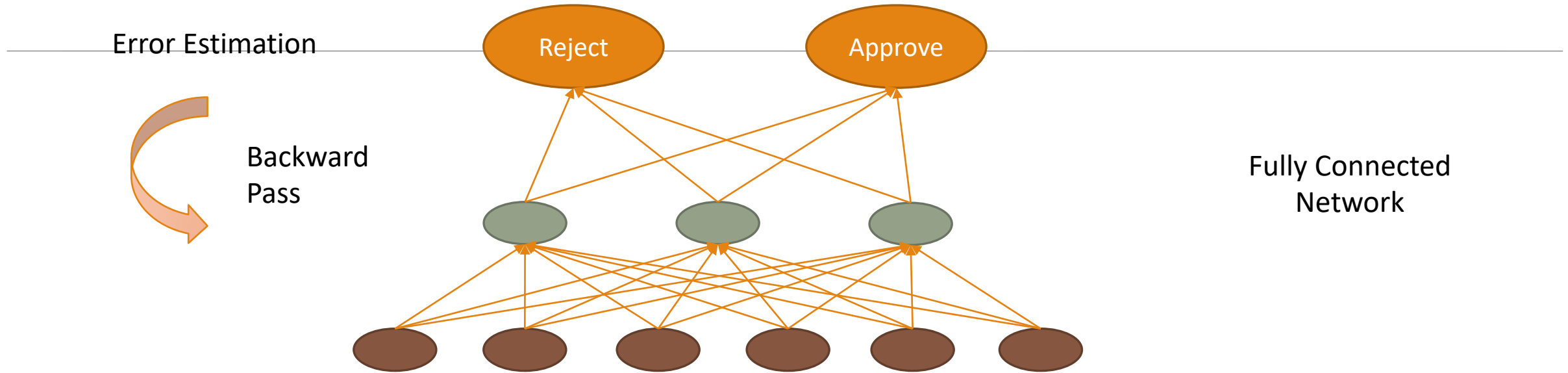
Neural Network: **Forward Pass**



- Step 1: Model initialization: Randomly set initialized neural network
- Step 2: Feed forward: The model is fed input data

CID	Credit Score	Income (10,000)	Term	Age	Gender	Number of Children	Status
1	6	2	1	35	0	0	Reject
2	7.5	5	3	38	0	2	Approve
3	4	3.5	2	23	1	4	Reject
...							
100000	8	2.5	1	35	0	1	Approve

Neural Network: Backward Pass



- Step 3: Calculate the error: How far is your model output from the actual value.
- Step 4: Compute gradient of the error: apply the chain rule to compute gradient with respect to the parameters
- Step 5: Update the parameters: Update the parameters (weights and biases) until the error becomes minimum.

Overview

- A Multi-Layer Neural Network
- Cost-benefit Analysis

Kicked Vehicle

- One of the biggest challenges of an auto dealership purchasing a used car at an auto auction is the risk of that the vehicle might have serious issues that prevent it from being sold to customers. The auto community calls these unfortunate purchases "kicks".
- Kicked cars often result when there are tampered odometers, mechanical issues the dealer is not able to address, issues with getting the vehicle title from the seller, or some other unforeseen problem. Kick cars can be very costly to dealers after transportation cost, throw-away repair work, and market losses in reselling the vehicle.

Kicked Vehicle

- Figure out which cars have a higher risk of being kick can provide real value to dealerships trying to provide the best inventory selection possible to their customers.
- Problem: Predict if the car purchased at the Auction is a Kick (bad buy).

Steps to Build A Multi-Layer Neural Network

- Step 1: Kicked Vehicle Data
- Step 2: Data Preprocessing
- Step 3: Build a Neural Network on Training Data
- Step 4: Evaluate Neural Network Model Performance

Step 1: Kicked Vehicle Data

	Auction	Color	IsBadBuy	MMRCurrentAuctionAveragePrice	Size	TopThreeAmericanName	VehBCost	VehicleAge	VehOdo	WarrantyCost	WheelType
0	ADESA	WHITE	No	2871	LARGE TRUCK	FORD	5300	8	75419	869	Alloy
1	ADESA	GOLD	Yes	1840	VAN	FORD	3600	8	82944	2322	Alloy
2	ADESA	RED	No	8931	SMALL SUV	CHRYSLER	7500	4	57338	588	Alloy
3	ADESA	GOLD	No	8320	CROSSOVER	FORD	8500	5	55909	1169	Alloy
4	ADESA	GREY	No	11520	LARGE TRUCK	FORD	10100	5	86702	853	Alloy
...	...	...	...	...	...	...	...	...	...	...	...
9995	ADESA	RED	No	7536	SMALL SUV	CHRYSLER	6600	4	85377	983	Alloy
9996	ADESA	BLACK	No	4921	LARGE TRUCK	GM	7000	7	89665	1543	Alloy
9997	ADESA	BLACK	No	9263	MEDIUM SUV	CHRYSLER	9000	4	59383	1417	Alloy
9998	ADESA	BLUE	No	3240	MEDIUM	OTHER	5500	4	48642	482	Covers
9999	ADESA	SILVER	No	6148	MEDIUM SUV	FORD	7000	5	82563	1243	Alloy

10000 rows x 11 columns

Target variable

Steps to build a decision tree model

- Step 1: Kicked Vehicle Data
- **Step 2: Data Preprocessing**
- Step 3: Build a Neural Network on Training Data
- Step 4: Evaluate Neural Network Model Performance

Step 2: Data Preprocessing

■ Dummy coding

- Dummy coding: convert categorical variables into numeric format
 - Create **dummy variables** to replace the original categorical variable
 - Dummy coding for a binary variable: gender

$$\text{male} = \begin{cases} 1 & \text{if } x = \text{male} \\ 0 & \text{otherwise} \end{cases}$$

Auction provider: ADESA, MANHEIM, OTHER

n categories, *n*-1 dummy variables

	Auction
Vehicle 1	OTHER
Vehicle 2	MANHEIM
Vehicle 3	ADESA



	Auction_MANHEIM	Auction_OTHER
Vehicle 1	0	1
Vehicle 2	1	0
Vehicle 3	0	0

Step 2: Data Preprocessing

■ Data Transformation

■ Standardization (Z-score standardization)

$$x_{\text{stand}} = \frac{x - \text{mean}(x)}{\text{standard deviation}(x)}$$

Numeric values will be rescaled to ensure the mean and the standard deviation to be 0 and 1, respectively

■ Normalization (max-min normalization)

$$x_{\text{norm}} = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Numeric values will be rescaled to values between 0 and 1

Step 2: Data Preprocessing

Target variable

	IsBadBuy	MMRCurrentAuctionAveragePrice	VehBCost	VehicleAge	VehOdo	WarrantyCost	Auction_MANHEIM	Auction_OTHER	Color_BLACK	...	WheelType_unkwnWheel
0	No	-1.337561	-0.809831	2.226463	0.264027	-0.674806	0	0	0		0
1	Yes	-1.762502	-1.779205	2.226463	0.776364	1.711088	0	0	0		0
2	No	1.160153	0.444654	-0.106683	-0.967013	-1.136221	0	0	0		0
3	No	0.908320	1.014874	0.476603	-1.064306	-0.182192	0	0	0		0
4	No	2.227245	1.927226	0.476603	1.032227	-0.701079	0	0	0		0
...	...	...	...	...	...	...	...	...	...	...	...
9995	No	0.585184	-0.068545	-0.106683	0.942014	-0.487612	0	0	0		0
9996	No	-0.492625	0.159543	1.643176	1.233962	0.431934	0	0	1		0
9997	No	1.296991	1.299984	-0.106683	-0.827780	0.225036	0	0	1		0
9998	No	-1.185473	-0.695787	-0.106683	-1.559078	-1.310278	0	0	0		0
9999	No	0.013100	0.159543	0.476603	0.750424	-0.060680	0	0	0		0

10000 rows x 40 columns

Step 2: Data Preprocessing

Split the data to training and testing sets

	IsBadBuy	MMRCurrentAuctionAveragePrice	VehBCost	VehicleAge	VehOdo	WarrantyCost	Auction_MANHEIM	Auction_OTHER	Color_BLACK	...	WheelType_unkwnWheel	
0	No	-1.337561	-0.809831	2.226463	0.264027	-0.674806	0	0	0		0	
1	Yes	-1.762502	-1.779205	2.226463	0.776364	1.711088	0	0	0		0	
2	No	1.160153	0.444654	-0.106683	-0.967013	-1.136221	0	0	0		0	
3	No	0.908320	1.014874	0.476603	-1.064306	-0.182192	0	0	0		0	
4	No	2.227245	1.927226	0.476603	1.032227	-0.701079	0	0	0		0	
...	...	...	...	...	...	...	...	...	...	...	...	
6995	No	-1.382487	-1.445626	-0.106683	0.874066	-0.060680	0	1	0		0	
6996	Yes	-0.400712	1.131769	0.476603	0.616774	0.152786	0	1	0		0	
6997	No	-1.320662	-1.274560	1.059890	0.418375	0.179059	0	1	0		0	
6998	No	-0.474490	-0.128418	-0.106683	-0.327493	-0.426857	0	1	0		0	
6999	No	-1.398149	-1.559670	1.059890	1.003155	-0.060680	0	1	0		0	
...	...	...	...	...	...	...	...	...	...	...	...	
9995	No	0.585184	-0.068545	-0.106683	0.942014	-0.487612	0	0	0		0	
9996	No	-0.492625	0.159543	1.643176	1.233962	0.431934	0	0	1		0	
9997	No	1.296991	1.299984	-0.106683	-0.827780	0.225036	0	0	1		0	
9998	No	-1.185473	-0.695787	-0.106683	-1.559078	-1.310278	0	0	0		0	
9999	No	0.013100	0.159543	0.476603	0.750424	-0.060680	0	0	0		0	

Training data

Testing data

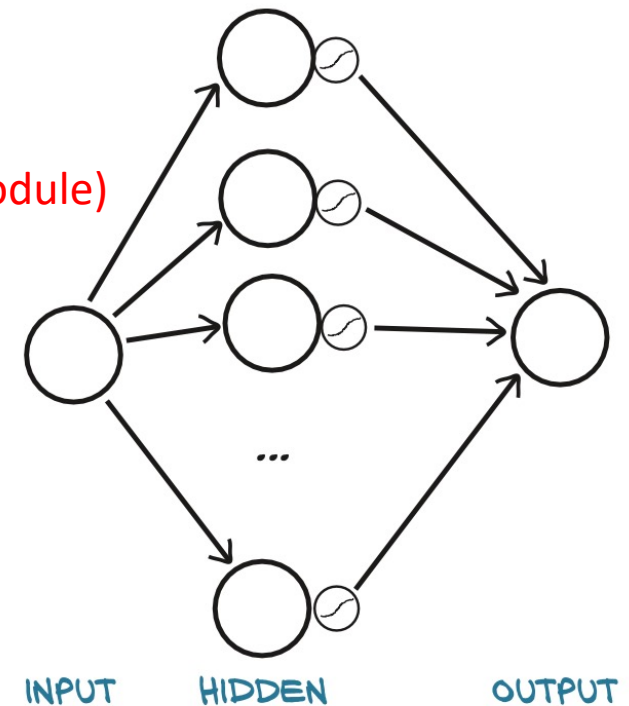
Steps to build a decision tree model

- Step 1: Kicked Vehicle Data
- Step 2: Data Preprocessing
- **Step 3: Build a Neural Network on Training Data**
- Step 4: Evaluate Neural Network Model Performance

Step 3: A Multi-Layer Neural Network

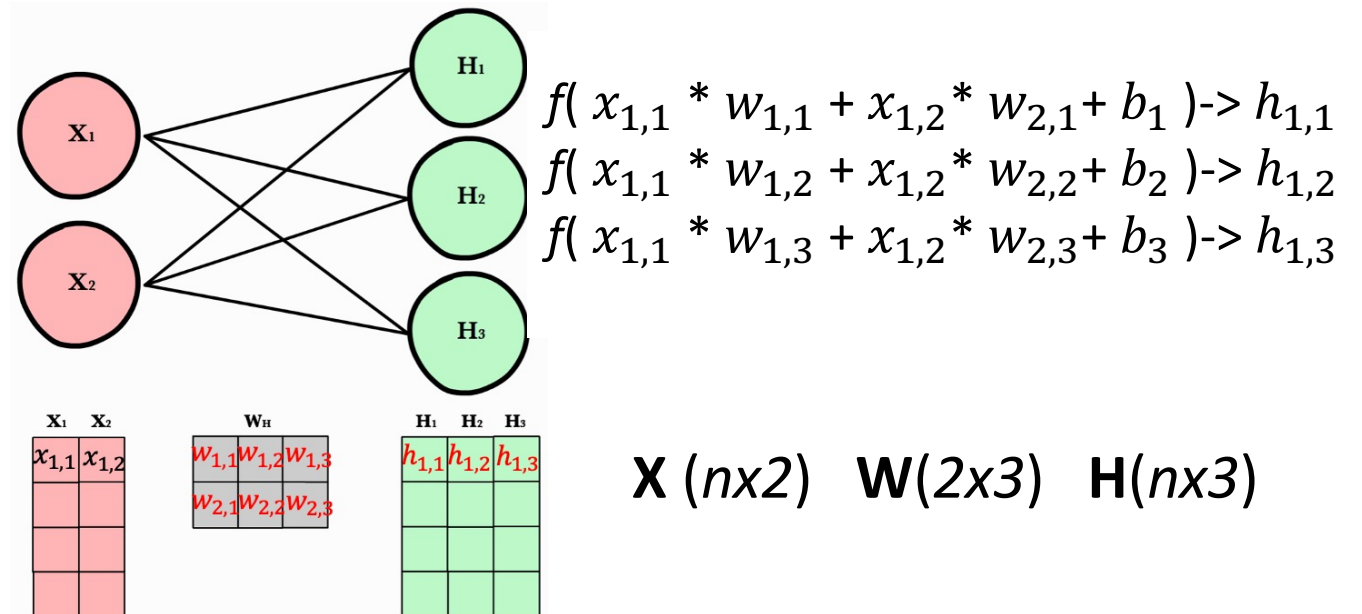
■ Build a model with nn.Sequential

```
class neural_network(nn.Module):  
    def __init__(self, in_size, hidden_size, out_size):  
        super().__init__() → Base class for all neural network modules  
        self.network = nn.Sequential(  
            nn.Linear(in_size, hidden_size), → Linear transformation  
            nn.ReLU(), → Activation function  
            nn.Linear(hidden_size, out_size)) → Linear transformation  
  
    def forward(self, x):  
        out = self.network(x)  
        return out
```



Step 3: A Multi-Layer Neural Network

- Build a model with `nn.Sequential`
 - The `Torch.nn` module: it contains the building blocks needed to create all sorts of neural network architectures.
 - A module can also have one or more submodules (subclasses of `nn.Module`) as attributes, and it will be able to track their parameters.
- `nn.Sequential`: A sequential container. Modules will be added to it in the order they are passed in the constructor
- `nn.Linear()`: Applies a linear transformation to the incoming data: $y = xW + b$



Step 3: A Multi-Layer Neural Network

■ Encode label to numeric

```
label_encoder = preprocessing.LabelEncoder()  
new_label = label_encoder.fit_transform(target_train)  
new_label[:5], target_train[:5]
```

```
(array([0, 0, 0, 1, 0]), 7681      No  
9031      No  
3691      No  
202       Yes  
5625      No  
Name: IsBadBuy, dtype: category  
Categories (2, object): ['No', 'Yes'])
```

Step 3: A Multi-Layer Neural Network

■ Tensors, Tensor Dataset, and Data Loader

```
# Create tensors from pandas dataframe
predictors_train_tensor = torch.tensor(predictors_train.values)
target_train_tensor = torch.tensor(label_encoder.transform(target_train.values))
predictors_test_tensor = torch.tensor(predictors_test.values)
target_test_tensor = torch.tensor(label_encoder.transform(target_test.values))

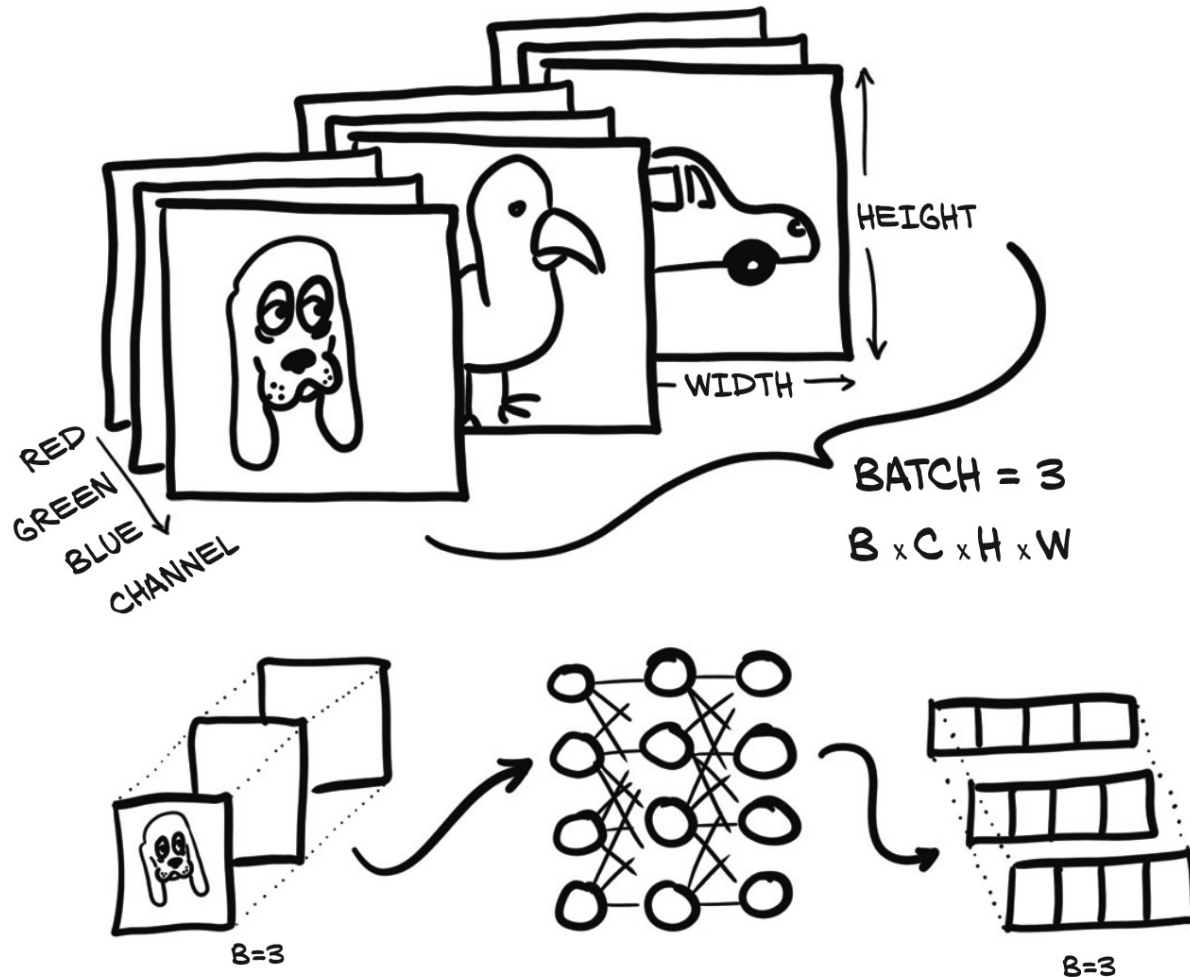
# Create tensor dataset (set target variable to long int type)
train_dataset = torch.utils.data.TensorDataset(predictors_train_tensor.float(), target_train_tensor.long())
test_dataset = torch.utils.data.TensorDataset(predictors_test_tensor.float(), target_test_tensor.long())

# Define training and testing data loader, and set batch size to 64
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=64, shuffle=False)
```

Step 3: A Multi-Layer Neural Network

- Tensors, Tensor Dataset, and Data Loader
 - **TensorDataset**: wraps tensors to form a tensor dataset. It stores the samples and their corresponding labels.
 - **DataLoader**: wraps an iterable around the Dataset to enable easy access to the samples.
 - Training with minibatches: The job of a data loader is to sample minibatches from a dataset, giving us the flexibility to choose from different sampling strategies.
 - **Batch size**: batch size is a hyperparameter that defines the number of samples (data points) to work through before updating the internal model parameters
 - **Epoch**: the number of epochs is a hyperparameter that defines the number times that the learning algorithm will work through the entire training dataset

Step 3: A Multi-Layer Neural Network



■ Batch size

■ $B \times C \times H \times W$

■ The reason we want to do this batching is multifaceted

- Less memory usage (compare to loading the whole data set)
- GPUs are highly parallelized
- Statistical information from the entire batch

Step 3: A Multi-Layer Neural Network

- Define a complete training loop

FOR N EPOCHS:

SPLIT DATASET IN MINIBATCHES

FOR EVERY MINIBATCH:

WITH EVERY SAMPLE IN MINIBATCH:

EVALUATE MODEL (FORWARD)

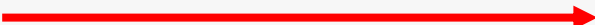
COMPUTE LOSS

ACCUMULATE GRADIENT OF LOSS (BACKWARD)

UPDATE MODEL WITH ACCUMULATED GRADIENT

Step 3: A Multi-Layer Neural Network

■ Define a complete training loop

```
def training_loop(n_epochs, optimizer, model, loss_fn, train_loader):  
    for epoch in range(0, n_epochs):  
        # Training Phase  
        model.train()  Set the model to training phase  
        loss_train = 0.0  
        for inputs, labels in train_loader:  Training with minibatches  
  
            outputs = model(inputs)  Forward pass  
  
            loss = loss_fn(outputs, labels)  Calculate loss  
  
            optimizer.zero_grad()  
            loss.backward()  Compute the gradient of current tensor  
            optimizer.step()  Update parameters  
            loss_train += loss.item()  
  
    if epoch == 0 or epoch == n_epochs-1 or epoch % 2000 == 0:  
        print('Epoch {}, Training loss {}'.format(epoch, loss_train / len(train_loader)))
```


Step 3: A Multi-Layer Neural Network

■ Model training

```
# Model training
torch.manual_seed(0)
model = neural_network(39, 64, 2)
optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss()

training_loop(n_epochs = 50, optimizer = optimizer, model = model, loss_fn = loss_fn, train_loader = train_loader)
```


Steps to build a decision tree model

- Step 1: Kicked Vehicle Data
- Step 2: Data Preprocessing
- Step 3: Build a Neural Network on Training Data
- Step 4: Evaluate Neural Network Model Performance

Step 4: Evaluate Neural Network Model Performance

■ Model evaluation

```
# Define testing function
def test(model, train_loader, test_loader):
```

```
    # testing phase
```

```
    model.eval() → Set the model to evaluation phase
```

```
    predict_train = []
```

```
    predict_test = []
```

```
    labels_train = []
```

```
    labels_test = []
```

```
    with torch.no_grad(): → Model parameters will not be changed during the evaluations
```

```
        for inputs, labels in train_loader:
```

```
            outputs = model(inputs)
```

```
            index_, predicted = torch.max(outputs, dim=1)
```

```
            predict_train.append(predicted.tolist())
```

```
            labels_train.append(labels.tolist())
```

```
        for inputs, labels in test_loader:
```

```
            outputs = model(inputs)
```

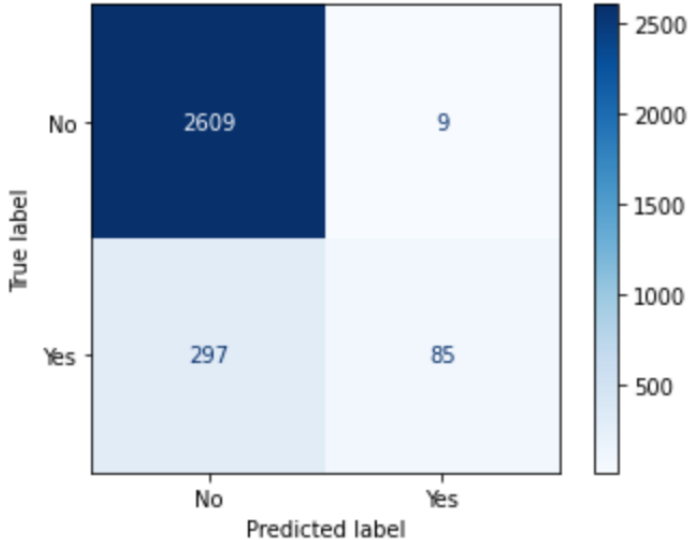
```
            index_, predicted = torch.max(outputs, dim=1)
```

```
            predict_test.append(predicted.tolist())
```

```
            labels_test.append(labels.tolist())
```

Step 4: Evaluate Neural Network Model Performance: Model Comparison

■ Comparing Decision Tree and Neural Network



Confusion matrix on test:
[[2547 71]
[284 98]]

Classification report on test:

	precision	recall	f1-score	support		precision	recall	f1-score	support
No	0.90	1.00	0.94	2618	0	0.90	0.97	0.93	2618
Yes	0.90	0.22	0.36	382	1	0.58	0.26	0.36	382
accuracy			0.90	3000				0.88	3000
macro avg	0.90	0.61	0.65	3000		0.74	0.61	0.65	3000
weighted avg	0.90	0.90	0.87	3000		0.86	0.88	0.86	3000

Overall performance
on NOT bad buy class

Overall performance
on bad buy class

Model's overall
performance

Step 4: Evaluate Neural Network Model Performance: Model Comparison

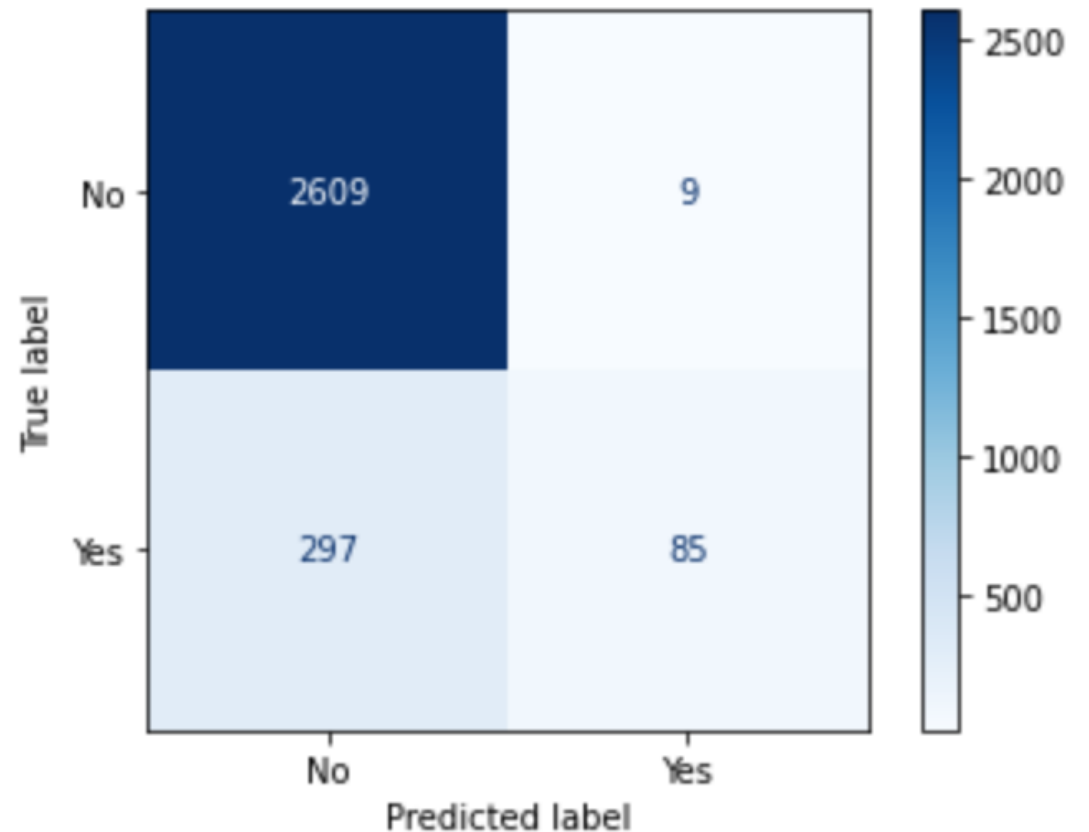
- Overall model performance comparison
 - Accuracy
- Compare performances on each class
 - F-measure: single metrics combines precision and recall and measures the overall performance on each class
 - Precision: confidence/effectiveness of predictions
 - Recall: ability of identifying instances belonging to a class

Step 4: Evaluate Neural Network Model Performance: Cost-benefit analysis

Assume the following costs / benefits:

- Cost of buying a bad car: \$2000
- Profit of purchasing a good car: \$300
- Opportunity cost of a good car that is missed: \$300

What is the total net benefit/cost of two models?



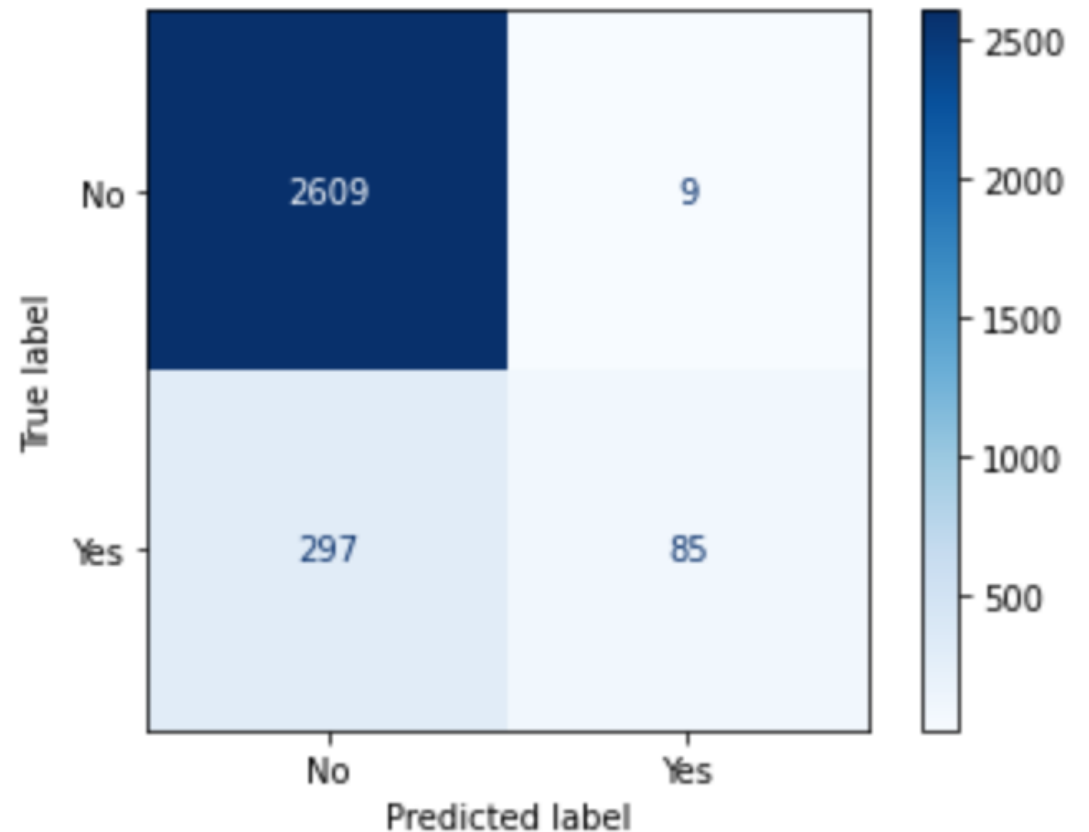
Step 4: Evaluate Neural Network Model Performance: Cost-benefit analysis

Assume the following costs / benefits:

- Cost of buying a bad car: \$2000
- Profit of purchasing a good car: \$300
- Opportunity cost of a good car that is missed: \$300

Decision Tree model:

- Cost: $297 * 2000$
- Profit: $2609 * 300$
- Opportunity cost: $9 * 300$
- Profit – Cost - Opportunity cost: 186,000



Exam Review

Exam Questions (120 minutes)

- 20 questions
 - 15 multiple choices and True false questions (30 points)
 - 4 short answer questions (35 points)
 - Business problem solving
 - Neural network architecture
 - The learning process of a neural network
 - Evaluation results (evaluation metrics, generalization, model comparison, cost-benefit analysis)
 - 1 code writing (10 points)

■ Quiz 1 questions

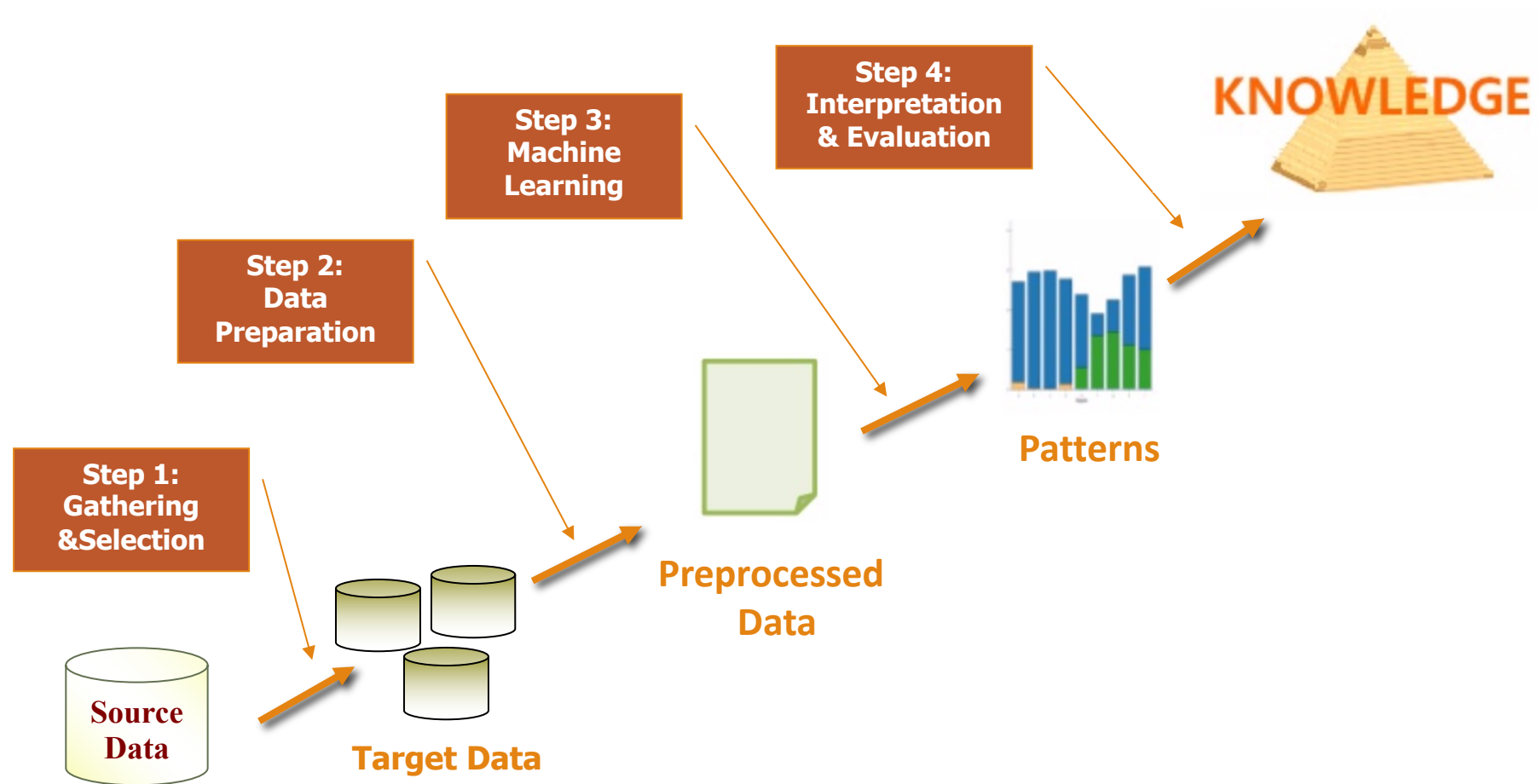
1. Introduction to Deep Learning

- Lecture 1&3 and lab 1&3
- Multiple choices and True False questions

Machine Learning

- Machine learning is a branch of artificial intelligence based on the idea that systems can learn from data, identify patterns and make decisions with minimal human intervention.
- Deep learning (also known as deep structured learning, hierarchical learning or deep machine learning) is a class of machine learning algorithms.
 - Deep learning algorithms: neural networks

Machine Learning Process



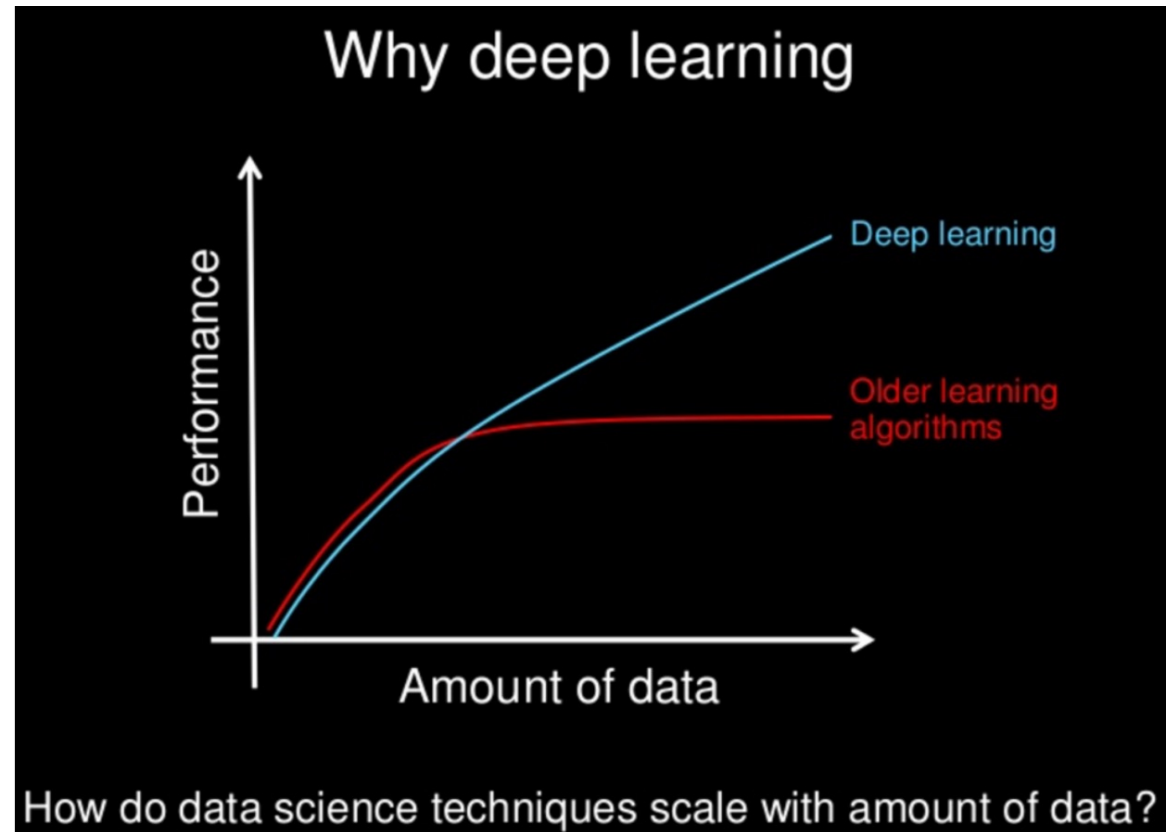
Why Deep Learning

■ Advantages:

- State of the art performance
- Capability of handling unstructured data
- Automatic feature engineering

■ Disadvantages:

- More data
- Bigger models; hard for model training and tuning
- Difficult to interpret
- Resources demanding



Why Deep Learning

- Different types of data

Class Roster			
Course:	MGT 500 Business Policy	Semester:	Spring 2010
Section:	2		
<u>Name</u>	<u>ID</u>	<u>Major</u>	<u>GPA</u>
Baker, Kenneth D.	324917628	MGT	2.9
Doyle, Joan E.	476193248	MKT	3.4
Finkle, Clive R.	548429344	PRM	2.8
Lewis, John C.	551742186	MGT	3.7
McFerran, Debra R.	409723145	IS	2.9
Sisneros, Michael	392416582	ACCT	3.3

Structured data: data stored in tabular form



Unstructured data: image, video, audio, text

Pretrained Networks

- Pretrained Network refer to a neural network that has been previously trained on a dataset and learned to extract powerful and informative features.
- It is common for researchers to publish their source code along with their papers, and often the code comes with weights that were obtained by training a model on a reference dataset.
- Using one of these models could enable us to, for example, equip our next web service with image-recognition capabilities with very little effort.

Pretrained Networks

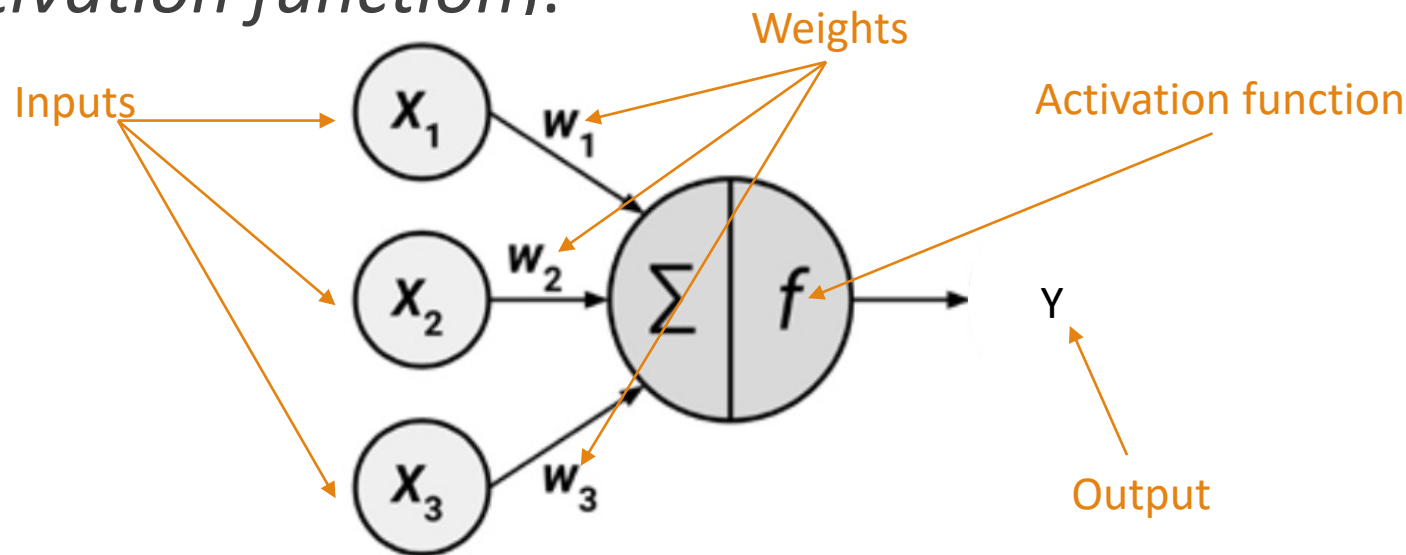
- Benefits of pretrained networks:
 - Easy to implement
 - Good performance (as a starting point)
 - Labeled data is not required
 - No model training process
- Disadvantages:
 - Not suitable for a different data set or task
 - Different data preprocessing
 - Require transfer and fine tune

2. Deep Learning Fundamentals

- Lecture 4 and lab 4
- Multiple choices and True False questions
- Short answer question (Neural network architecture)

Artificial Neural Network

- The basic building block of these complicated functions is the *neuron*.
 - A linear transformation of the input (for example, multiplying the input by a number [the *weight*] and adding a constant [the *bias*]) followed by the application of a fixed nonlinear function (referred to as the *activation function*).



$$o = f(w * x + b)$$

Artificial Neural Network

- The simplest unit in (deep) neural networks is a linear operation followed by an activation function.
- The **activation function** plays two important roles
 - It allows the output function to **have different slopes at different values**; neural networks can approximate arbitrary functions
 - At the last layer of the network, it has the role **of concentrating the outputs of the preceding linear operation into a given range** (e.g., Sigmoid and Softmax functions)
- Softmax function:
$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Category	Output	e^{z_i}	Softmax
Reject	-1	0.368	0.047
Approve	2	7.39	0.953

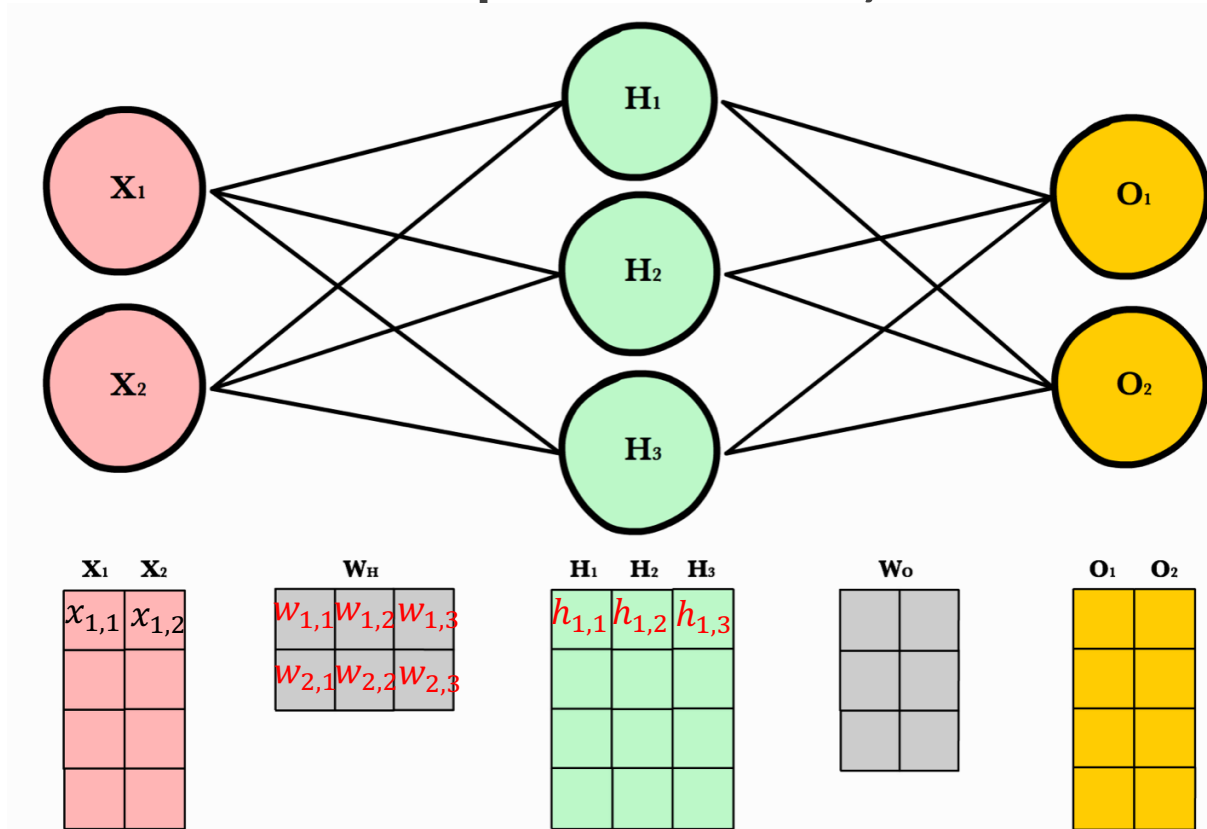
Artificial Neural Network

- A network with 2 input nodes, 3 hidden nodes, and 2 output nodes.

Input layer

Hidden layer

Output layer



$$f(x_{1,1} * w_{1,1} + x_{1,2} * w_{2,1} + b_1) \rightarrow h_{1,1}$$

$$f(x_{1,1} * w_{1,2} + x_{1,2} * w_{2,2} + b_2) \rightarrow h_{1,2}$$

$$f(x_{1,1} * w_{1,3} + x_{1,2} * w_{2,3} + b_3) \rightarrow h_{1,3}$$

$$\mathbf{X} (n \times 2) \quad \mathbf{W} (2 \times 3) \quad \mathbf{H} (n \times 3)$$

3-dimensional hidden space

From <https://ml-cheatsheet.readthedocs.io/en/latest/forwardpropagation.html>

Fundamental Building Block: Tensors

- **Neural network: a function describing how inputs are mapped to the outputs**

A NEURAL NETWORK

$$O = \tanh(w_n(\dots \tanh(w_2(\tanh(w_1 x + b_1) + b_2)) \dots + b_n))$$

Diagram illustrating a neural network function. The input x is fed into the first layer. The output of the first layer is passed to the second layer, and so on, until the final output O is produced. The weights w_i and biases b_i are labeled as learned parameters.

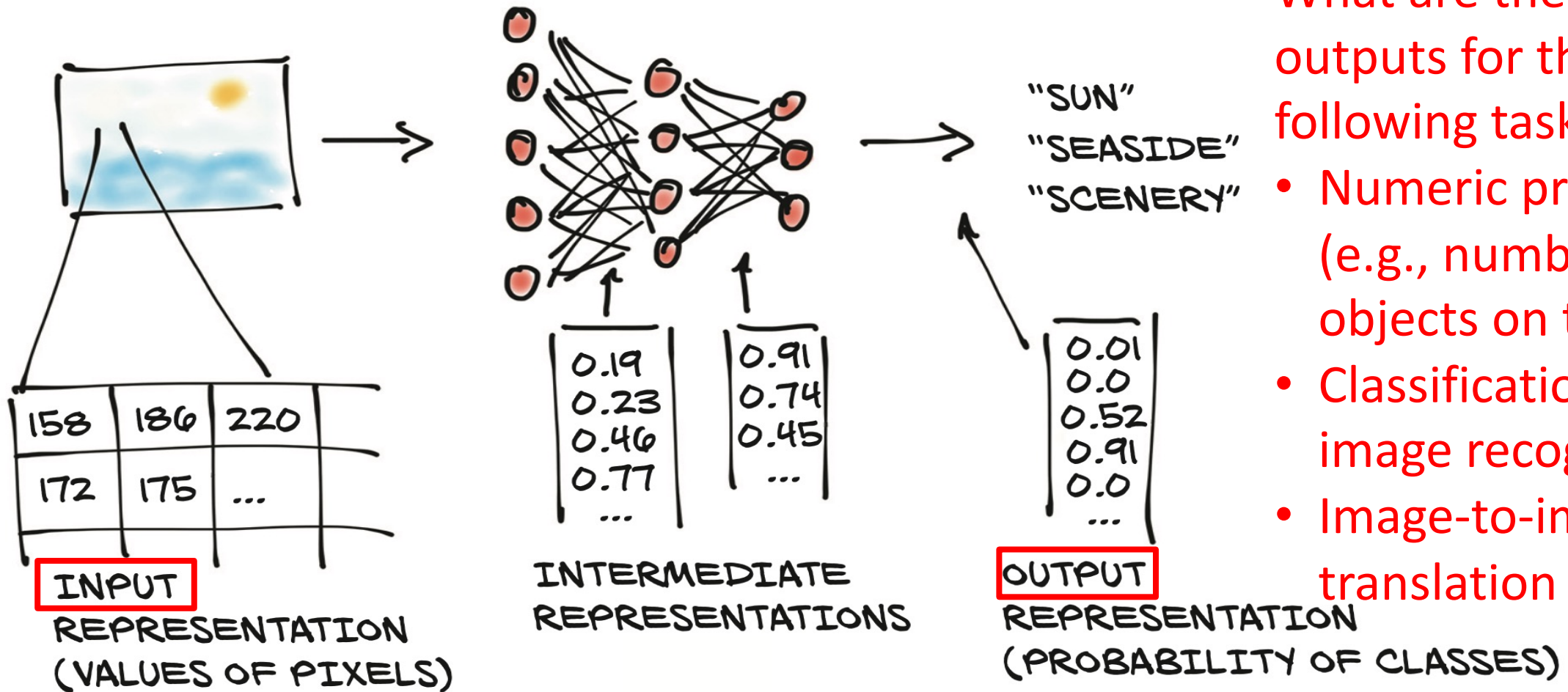
- **Tensors** refer to the generalization of vectors and matrices to an arbitrary number of dimensions
 - Transpose A Tensor
 - Transfer Tensor to GPU
 - Represent Tabular Data with Tensors

Tensor Element Types

- `torch.float32` or `torch.float`: 32-bit floating-point
- `torch.float64` or `torch.double`: 64-bit, double-precision floating-point
- `torch.float16` or `torch.half`: 16-bit, half-precision floating-point
- `torch.int8`: signed 8-bit integers
- `torch.uint8`: unsigned 8-bit integers
- `torch.int16` or `torch.short`: signed 16-bit integers
- `torch.int32` or `torch.int`: signed 32-bit integers
- `torch.int64` or `torch.long`: signed 64-bit integers
- `torch.bool`: **Boolean**

The default data type
for tensors is 32-bit
floating-point

Artificial Neural Network

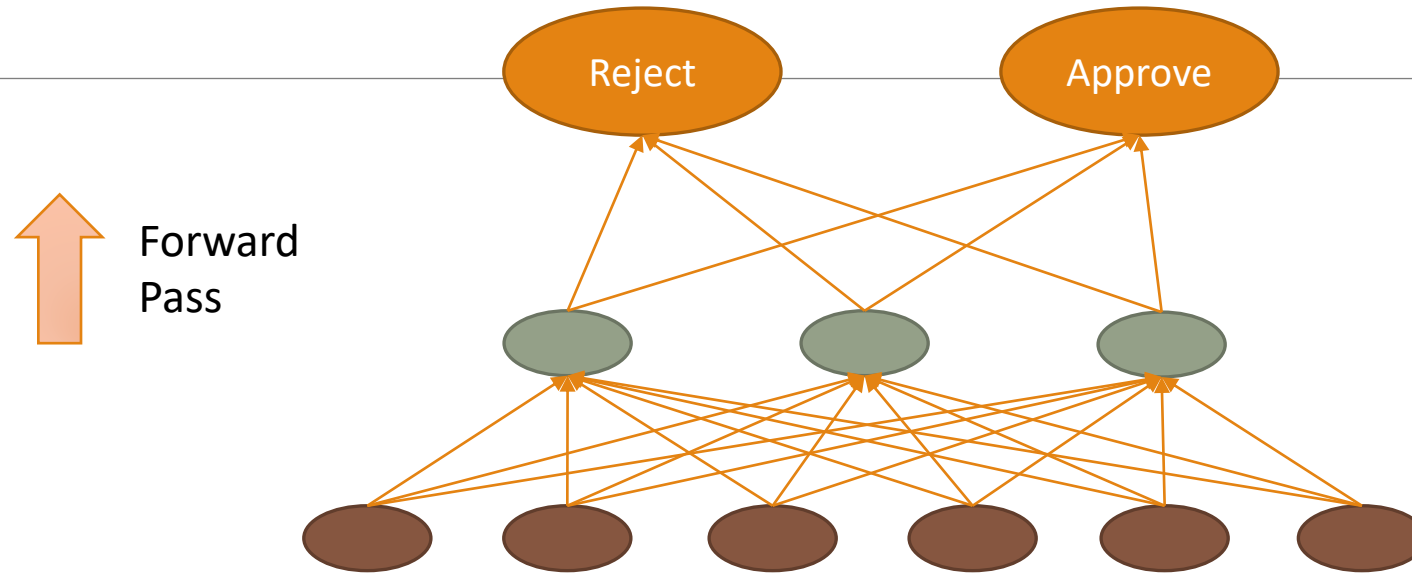


- What are the model outputs for the following tasks?
- Numeric prediction (e.g., number of objects on the image)
 - Classification (e.g., image recognition)
 - Image-to-image translation

3. Using a Neural Network to Fit the Data

- Lecture 5 and lab 5
- Multiple choices and True False questions
- Short answer question (The learning process of a neural network)

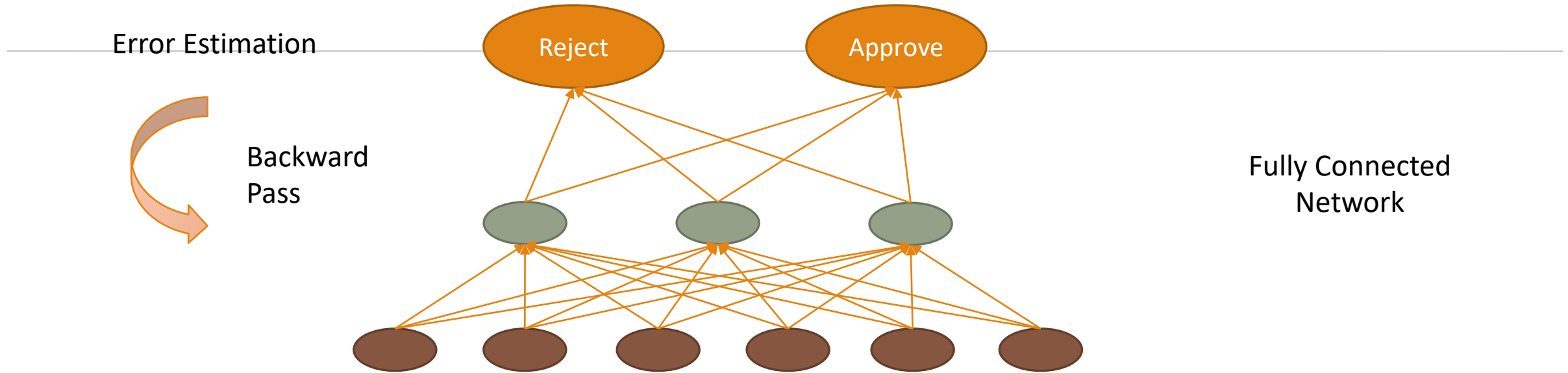
Neural Network: Forward Pass



- Step 1: Model initialization: Randomly set initialized neural network
- Step 2: Feed forward: The model is fed input data

CID	Credit Score	Income (10,000)	Term	Age	Gender	Number of Children	Status
1	6	2	1	35	0	0	Reject
2	7.5	5	3	38	0	2	Approve
3	4	3.5	2	23	1	4	Reject
...							
100000	8	2.5	1	35	0	1	Approve

Neural Network: Backward Pass



- Step 3: Calculate the error: How far is your model output from the actual value.
- Step 4: Compute gradient of the error: apply the chain rule to compute gradient with respect to the parameters
- Step 5: Update the parameters: Update the parameters (weights and biases) until the error becomes minimum.

The Learning Process

- **Forward pass**

- Given input data and initial values for the weights, the model is fed input data

- **Backward pass**

- A measure of the error is evaluated by comparing the outputs to the ground truth
 - The gradient of the error with respect to the parameters is computed using the chain rule
 - Optimize the parameter of the model: the value of the weights is then updated in the direction that leads to a decrease in the error.
- The procedure is repeated until the error falls below an acceptable level.

The Learning Process

Backward pass: 1. A measure of the error is evaluated by comparing the outputs to the ground truth

- **Goal—minimize the error:** Our optimization process should therefore aim at finding w and b so that the loss function is at a minimum (the predicted t_c is close to the actual t_c)
 - We need to exactly define a measure of the error
 - *loss function*—should be high if the error is high and should ideally be as low as possible for a perfect match

The Learning Process

- A **loss function** (or **cost function**) is a function that computes a single numerical value that the learning process will attempt to minimize
 - The calculation of loss typically involves taking the difference between the model outputs and actual values
 - Absolute error loss: $|t_p - t_c|$
 - Squared error loss: $(t_p - t_c)^2$
- Define the objective of learning

The Learning Process

Backward pass: 2. The gradient of the error with respect to the parameters is computed using the chain rule

- We compute the individual derivatives of the loss with respect to each parameter

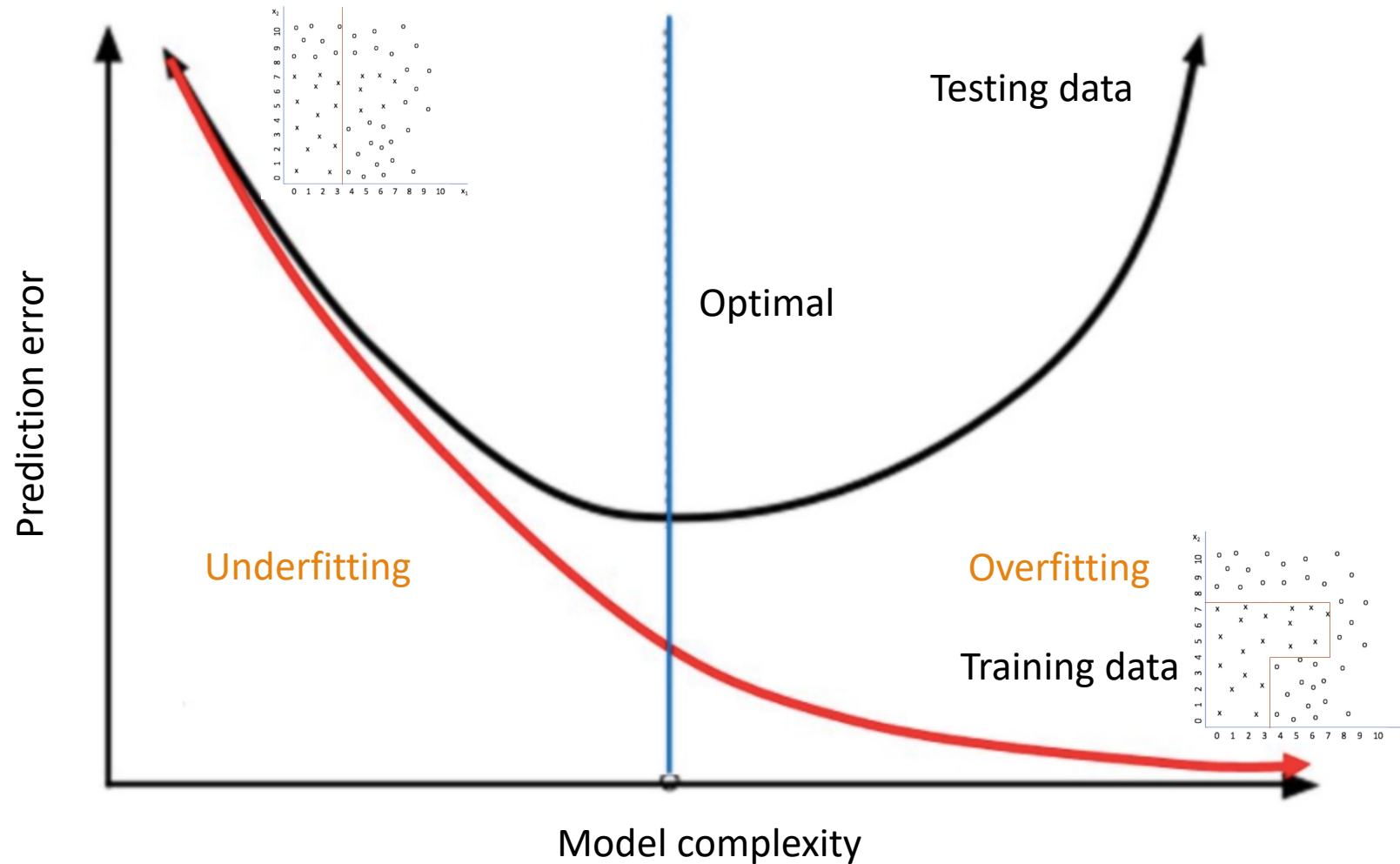
```
# Define the gradient function
def grad_fn(t_u, t_c, t_p, w, b):
    dloss_dtp = dloss_fn(t_p, t_c)
    dloss_dw = dloss_dtp * dmodel_dw(t_u, w, b)
    dloss_db = dloss_dtp * dmodel_db(t_u, w, b)
    return torch.stack([dloss_dw.mean(), dloss_db.mean()])
```

→ The gradient depends on both
→ the loss function and model

The Learning Process

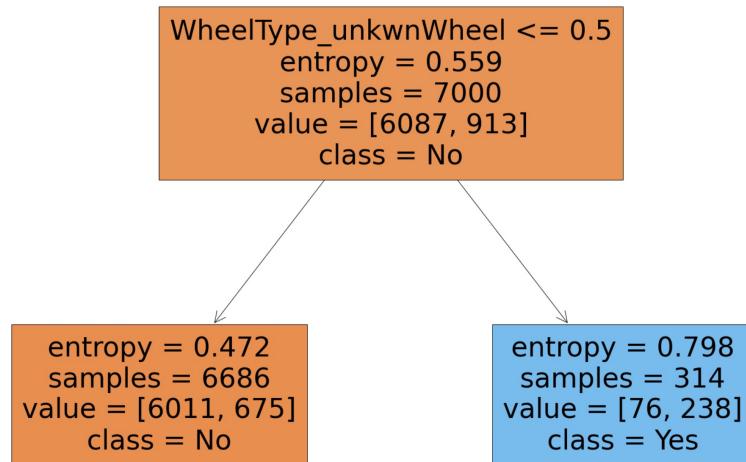
- **Backward pass:** 3. Optimize the parameter of the model: the value of the weights is then updated in the direction that leads to a decrease in the error.
- **Learning rate:** The learning rate is a hyperparameter that controls how much to change the model in response to the estimated error each time the model weights are updated
 - **The effect of learning rate**
 - **The effect of epoch**

Generalization and Overfitting



Generalization and Overfitting

- Comparing the model **performances** on **training** and **testing** data
 - Max depth = 1



Training performance

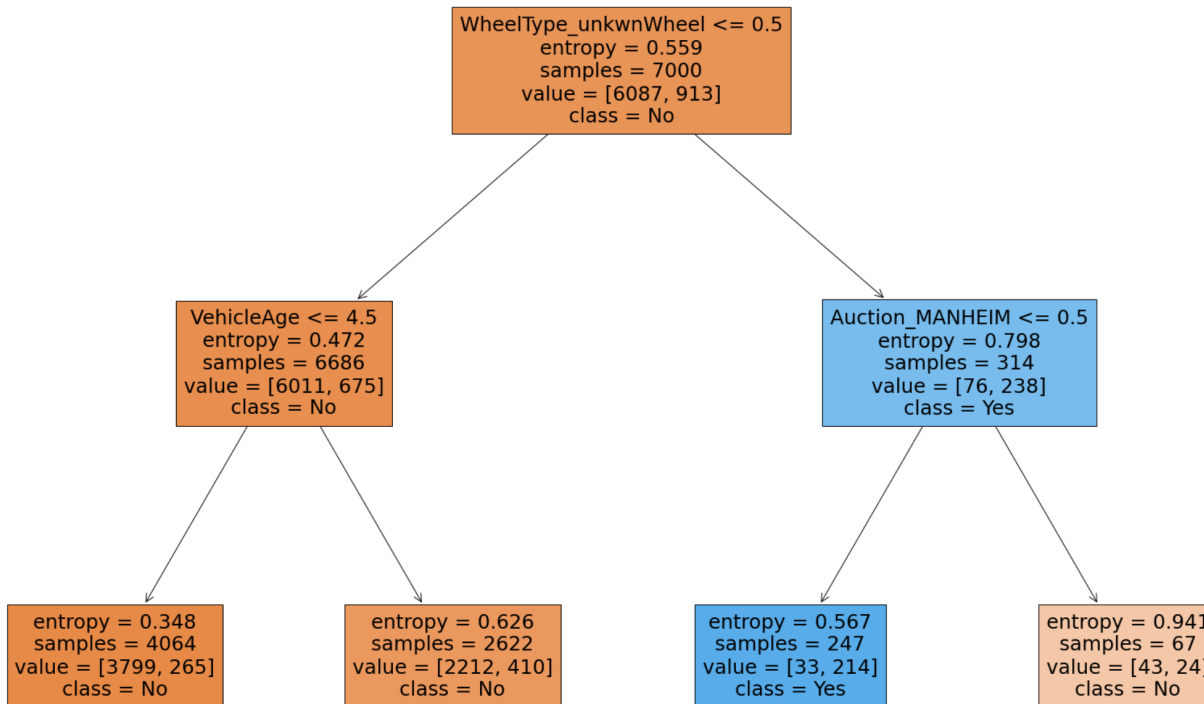
	precision	recall	f1-score	support
No	0.90	0.99	0.94	6087
Yes	0.76	0.26	0.39	913
accuracy			0.89	7000
macro avg	0.83	0.62	0.66	7000
weighted avg	0.88	0.89	0.87	7000

Testing performance

	precision	recall	f1-score	support
No	0.90	0.99	0.94	2618
Yes	0.76	0.26	0.39	382
accuracy			0.90	3000
macro avg	0.83	0.62	0.67	3000
weighted avg	0.88	0.90	0.87	3000

Generalization and Overfitting

- Comparing the model **performances** on **training** and **testing** data
 - Max depth = 2



Training performance

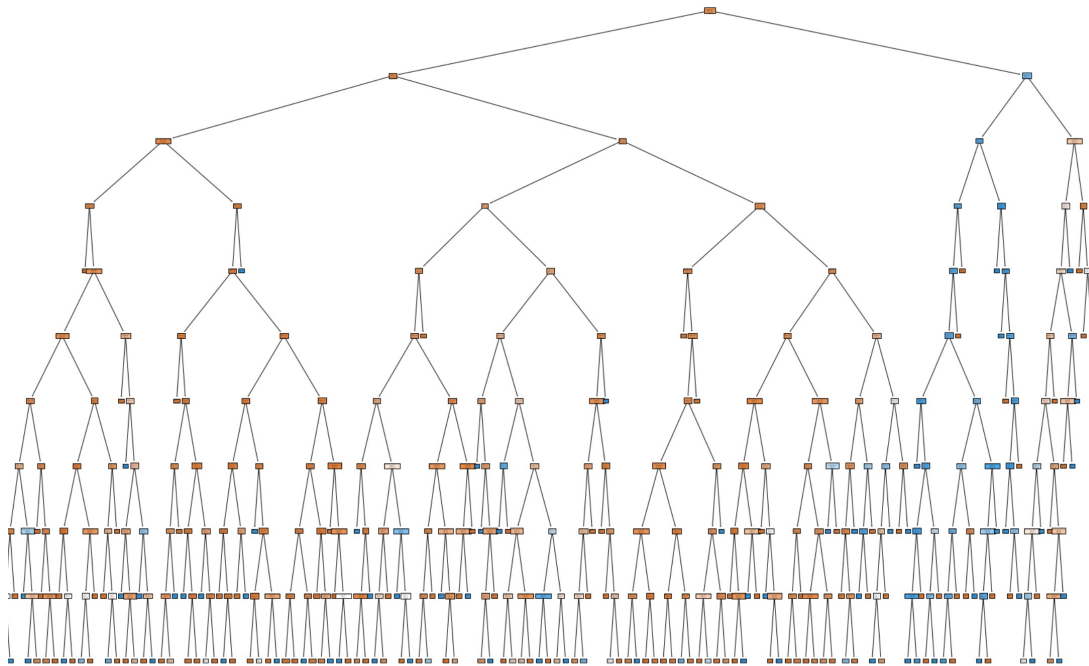
	precision	recall	f1-score	support
No	0.90	0.99	0.94	6087
Yes	0.87	0.23	0.37	913
accuracy			0.90	7000
macro avg	0.88	0.61	0.66	7000
weighted avg	0.89	0.90	0.87	7000

Testing performance

	precision	recall	f1-score	support
No	0.90	1.00	0.94	2618
Yes	0.90	0.22	0.36	382
accuracy			0.90	3000
macro avg	0.90	0.61	0.65	3000
weighted avg	0.90	0.90	0.87	3000

Generalization and Overfitting

- Comparing the model **performances** on **training** and **testing** data
 - Max depth = 10



Training performance

	precision	recall	f1-score	support
No	0.92	1.00	0.96	6087
Yes	0.94	0.41	0.57	913
accuracy			0.92	7000
macro avg	0.93	0.70	0.76	7000
weighted avg	0.92	0.92	0.90	7000

Testing performance

	precision	recall	f1-score	support
No	0.90	0.97	0.93	2618
Yes	0.56	0.25	0.35	382
accuracy			0.88	3000
macro avg	0.73	0.61	0.64	3000
weighted avg	0.86	0.88	0.86	3000

Generalization and Overfitting

Causes for Overfitting:

- Training data is not a good representation of testing (new) data
 - Insufficient training data
 - Noises in data: inconsistent class labels for the same values in feature set (input attributes)
 - Outliers in data: the number of samples with a given combination of class labels and feature values is small.
- An algorithm's inability to avoid overfitting noises/outliers or to train generalizable models via small amounts of training data
 - Complex model

Generalization and Overfitting

Avoidance of Overfitting

- Data strategies
 - Secure sufficient data
 - Identify and handle potential outliers and noises
- Evaluation strategies
 - **Identifying overfitting**: comparing the model **performance** on **training** and **testing** data
 - **Avoid big gap between the training and testing accuracy**
- Model strategies
 - Select proper algorithm and manage model complexity
 - Compare different algorithms
 - Lower model complexity via method-specific parameters

4. Classification

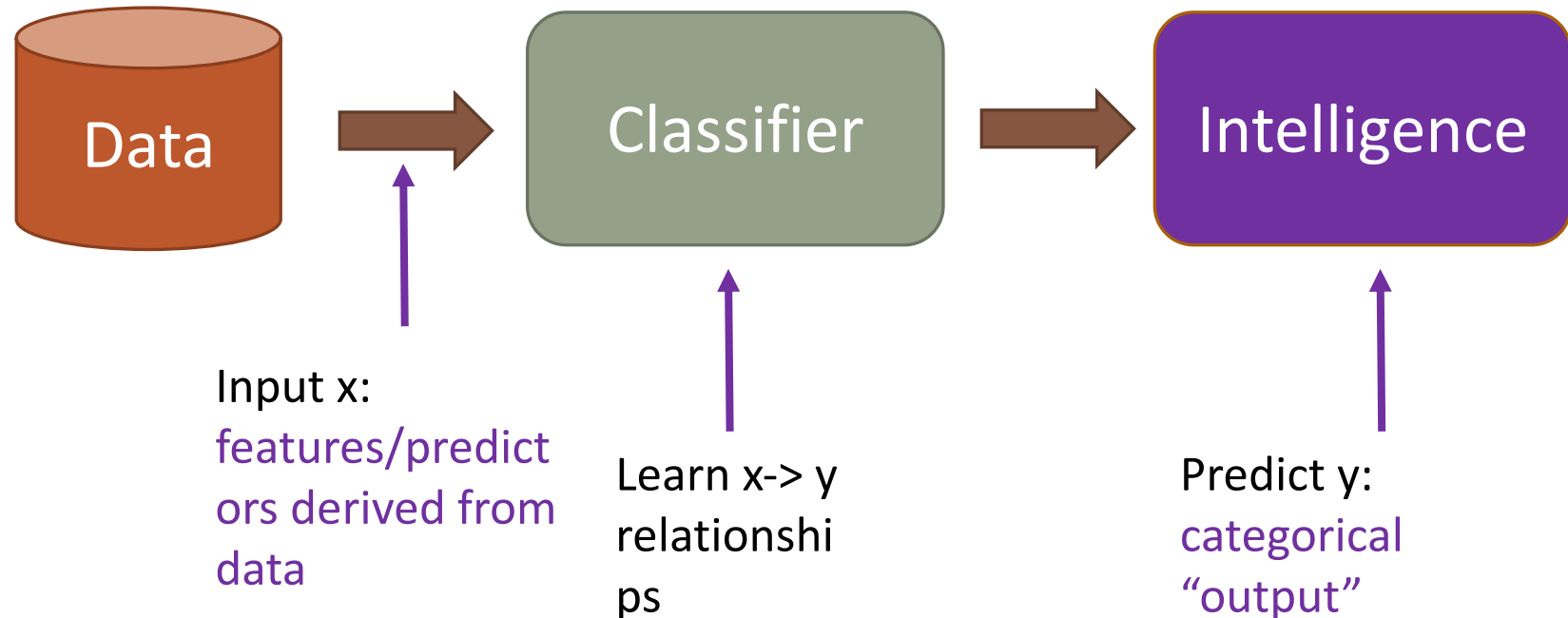
- Lecture 2&6 and lab 2&6
- Multiple choices and True False questions
- Short answer question: evaluation results (evaluation metrics, generalization, model comparison, cost-benefit analysis)
- Code writing

Classification

- Classify objects into a set of **pre-specified** classes (or categories) based on the values of relevant object variables (features).

Why classification?

Estimating/predicting the class or category of action to support **time and cost-effective decision making**.

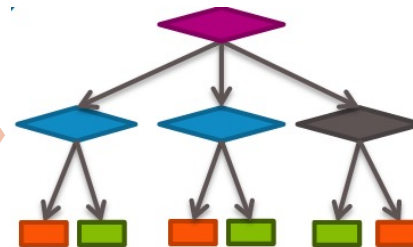


Classification

Information of Cars

- Auction
- Color
- MMRCurrentAuctionAveragePrice
- Size
- TopThreeAmericanName
- VehBCost
- VehicleAge
- VehOdo
- WarrantyCost
- WheelType

Predictors (X)



Classifier

IsBadBuy

Target variable
/Label (Y)

Classification

- **Predictor:** A predictor (or **predictor variable**) is a variable whose values will be used to predict the value of the target variable.
- **Target variable:** The target variable (or **outcome variable**) is the variable whose values are to be modeled and predicted by other variables.
- **Classifier:** an algorithm/classification model that implements classification (mapping input data to a category).

Step 1: Kicked Vehicle Data

	Auction	Color	IsBadBuy	MMRCurrentAuctionAveragePrice	Size	TopThreeAmericanName	VehBCost	VehicleAge	VehOdo	WarrantyCost	WheelType
0	ADESA	WHITE	No	2871	LARGE TRUCK	FORD	5300	8	75419	869	Alloy
1	ADESA	GOLD	Yes	1840	VAN	FORD	3600	8	82944	2322	Alloy
2	ADESA	RED	No	8931	SMALL SUV	CHRYSLER	7500	4	57338	588	Alloy
3	ADESA	GOLD	No	8320	CROSSOVER	FORD	8500	5	55909	1169	Alloy
4	ADESA	GREY	No	11520	LARGE TRUCK	FORD	10100	5	86702	853	Alloy
...	...	...	...	...	...	...	...	...	...	...	...
9995	ADESA	RED	No	7536	SMALL SUV	CHRYSLER	6600	4	85377	983	Alloy
9996	ADESA	BLACK	No	4921	LARGE TRUCK	GM	7000	7	89665	1543	Alloy
9997	ADESA	BLACK	No	9263	MEDIUM SUV	CHRYSLER	9000	4	59383	1417	Alloy
9998	ADESA	BLUE	No	3240	MEDIUM	OTHER	5500	4	48642	482	Covers
9999	ADESA	SILVER	No	6148	MEDIUM SUV	FORD	7000	5	82563	1243	Alloy

10000 rows x 11 columns

Target variable

Step 2: Data Preprocessing

■ Dummy coding

- Dummy coding: convert categorical variables into numeric format
 - Create **dummy variables** to replace the original categorical variable
 - Dummy coding for a binary variable: gender

$$\text{male} = \begin{cases} 1 & \text{if } x = \text{male} \\ 0 & \text{otherwise} \end{cases}$$

Auction provider: ADESA, MANHEIM, OTHER

n categories, *n*-1 dummy variables

	Auction
Vehicle 1	OTHER
Vehicle 2	MANHEIM
Vehicle 3	ADESA



	Auction_MANHEIM	Auction_OTHER
Vehicle 1	0	1
Vehicle 2	1	0
Vehicle 3	0	0

Step 2: Data Preprocessing

■ Data Transformation

■ Standardization (Z-score standardization)

$$x_{\text{stand}} = \frac{x - \text{mean}(x)}{\text{standard deviation}(x)}$$

Numeric values will be rescaled to ensure the mean and the standard deviation to be 0 and 1, respectively

■ Normalization (max-min normalization)

$$x_{\text{norm}} = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Numeric values will be rescaled to values between 0 and 1

Step 2: Data Preprocessing

Target variable

	IsBadBuy	MMRCurrentAuctionAveragePrice	VehBCost	VehicleAge	VehOdo	WarrantyCost	Auction_MANHEIM	Auction_OTHER	Color_BLACK	...	WheelType_unkwnWheel
0	No	-1.337561	-0.809831	2.226463	0.264027	-0.674806	0	0	0		0
1	Yes	-1.762502	-1.779205	2.226463	0.776364	1.711088	0	0	0		0
2	No	1.160153	0.444654	-0.106683	-0.967013	-1.136221	0	0	0		0
3	No	0.908320	1.014874	0.476603	-1.064306	-0.182192	0	0	0		0
4	No	2.227245	1.927226	0.476603	1.032227	-0.701079	0	0	0		0
...	...	...	...	...	...	...	...	...	...	...	...
9995	No	0.585184	-0.068545	-0.106683	0.942014	-0.487612	0	0	0		0
9996	No	-0.492625	0.159543	1.643176	1.233962	0.431934	0	0	1		0
9997	No	1.296991	1.299984	-0.106683	-0.827780	0.225036	0	0	1		0
9998	No	-1.185473	-0.695787	-0.106683	-1.559078	-1.310278	0	0	0		0
9999	No	0.013100	0.159543	0.476603	0.750424	-0.060680	0	0	0		0

10000 rows x 40 columns

Step 2: Data Preprocessing

Split the data to training and testing sets

	IsBadBuy	MMRCurrentAuctionAveragePrice	VehBCost	VehicleAge	VehOdo	WarrantyCost	Auction_MANHEIM	Auction_OTHER	Color_BLACK	...	WheelType_unkwnWheel	
0	No	-1.337561	-0.809831	2.226463	0.264027	-0.674806	0	0	0		0	
1	Yes	-1.762502	-1.779205	2.226463	0.776364	1.711088	0	0	0		0	
2	No	1.160153	0.444654	-0.106683	-0.967013	-1.136221	0	0	0		0	
3	No	0.908320	1.014874	0.476603	-1.064306	-0.182192	0	0	0		0	
4	No	2.227245	1.927226	0.476603	1.032227	-0.701079	0	0	0		0	
...	...	...	...	...	...	...	...	...	...	...	...	
6995	No	-1.382487	-1.445626	-0.106683	0.874066	-0.060680	0	1	0		0	
6996	Yes	-0.400712	1.131769	0.476603	0.616774	0.152786	0	1	0		0	
6997	No	-1.320662	-1.274560	1.059890	0.418375	0.179059	0	1	0		0	
6998	No	-0.474490	-0.128418	-0.106683	-0.327493	-0.426857	0	1	0		0	
6999	No	-1.398149	-1.559670	1.059890	1.003155	-0.060680	0	1	0		0	
...	...	...	...	...	...	...	...	...	...	...	...	
9995	No	0.585184	-0.068545	-0.106683	0.942014	-0.487612	0	0	0		0	
9996	No	-0.492625	0.159543	1.643176	1.233962	0.431934	0	0	1		0	
9997	No	1.296991	1.299984	-0.106683	-0.827780	0.225036	0	0	1		0	
9998	No	-1.185473	-0.695787	-0.106683	-1.559078	-1.310278	0	0	0		0	
9999	No	0.013100	0.159543	0.476603	0.750424	-0.060680	0	0	0		0	

Training data

Testing data

Step 3: A Multi-Layer Neural Network

■ Build a model with nn.Sequential

```
class neural_network(nn.Module):  
    def __init__(self, in_size, hidden_size, out_size):  
        super().__init__()   
        self.network = nn.Sequential(  
            nn.Linear(in_size, hidden_size),  
            nn.ReLU(),  
            nn.Linear(hidden_size, out_size))  
  
    def forward(self, x):  
        out = self.network(x)  
        return out
```

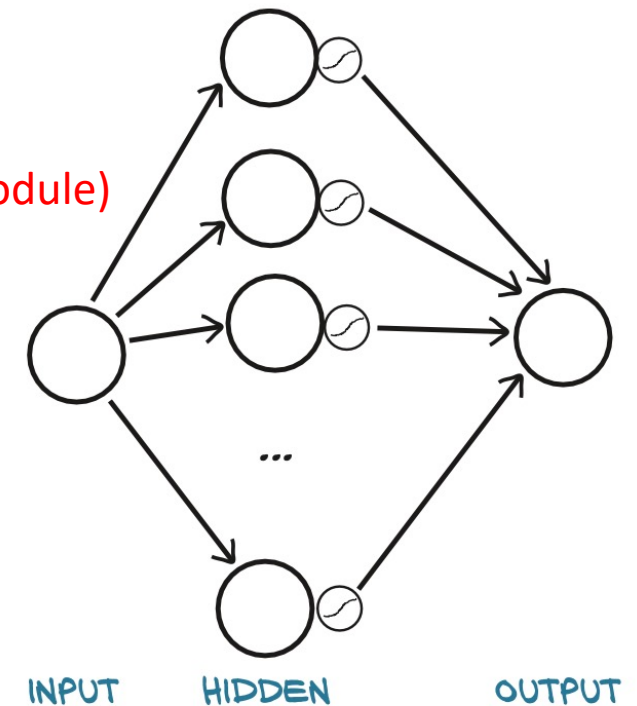
Base class for all neural network modules

Call the `__init__()` function from superclass(`nn.Module`)

Linear transformation

Activation function

Linear transformation



Step 3: A Multi-Layer Neural Network

■ Encode label to numeric

```
label_encoder = preprocessing.LabelEncoder()  
new_label = label_encoder.fit_transform(target_train)  
new_label[:5], target_train[:5]
```

```
(array([0, 0, 0, 1, 0]), 7681      No  
9031      No  
3691      No  
202       Yes  
5625      No  
Name: IsBadBuy, dtype: category  
Categories (2, object): ['No', 'Yes'])
```


Step 3: A Multi-Layer Neural Network

- Tensors, Tensor Dataset, and Data Loader
 - **TensorDataset**: wraps tensors to form a tensor dataset. It stores the samples and their corresponding labels.
 - **DataLoader**: wraps an iterable around the Dataset to enable easy access to the samples.
 - Training with minibatches: The job of a data loader is to sample minibatches from a dataset, giving us the flexibility to choose from different sampling strategies.
 - **Batch size**: batch size is a hyperparameter that defines the number of samples (data points) to work through before updating the internal model parameters
 - **Epoch**: the number of epochs is a hyperparameter that defines the number times that the learning algorithm will work through the entire training dataset

Step 3: A Multi-Layer Neural Network

- Define a complete training loop

FOR N EPOCHS:

SPLIT DATASET IN MINIBATCHES

FOR EVERY MINIBATCH:

WITH EVERY SAMPLE IN MINIBATCH:

EVALUATE MODEL (FORWARD)

COMPUTE LOSS

ACCUMULATE GRADIENT OF LOSS (BACKWARD)

UPDATE MODEL WITH ACCUMULATED GRADIENT

Step 4: Evaluate Neural Network Model Performance

- Holdout Evaluation

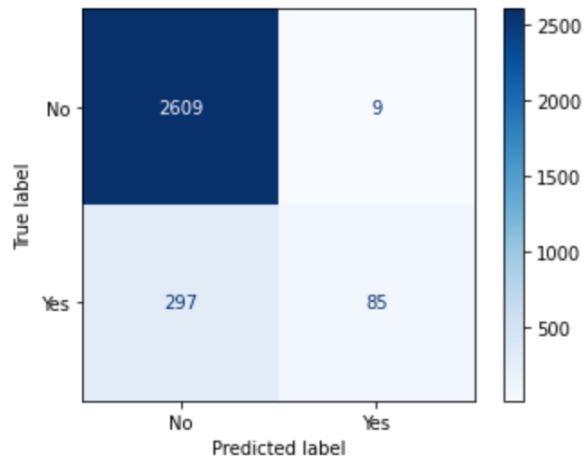
- Splitting method (percentage split)- divide into training and testing sets (e.g. 70%/30% or 2/3 to 1/3)
 - 2 partitions, e.g.
 - 70% and 30% in training and testing
 - Train the model on the **training data**
 - **Training and testing data are not overlapped**

Step 4: Evaluate Neural Network Model Performance

- **Understand the confusion matrix**
- **Understand the meaning of evaluation metrics**
- **Model comparison**
- **Cost-Benefit analysis**
- **Overfitting issue**

Step 4: Evaluate Neural Network Model Performance

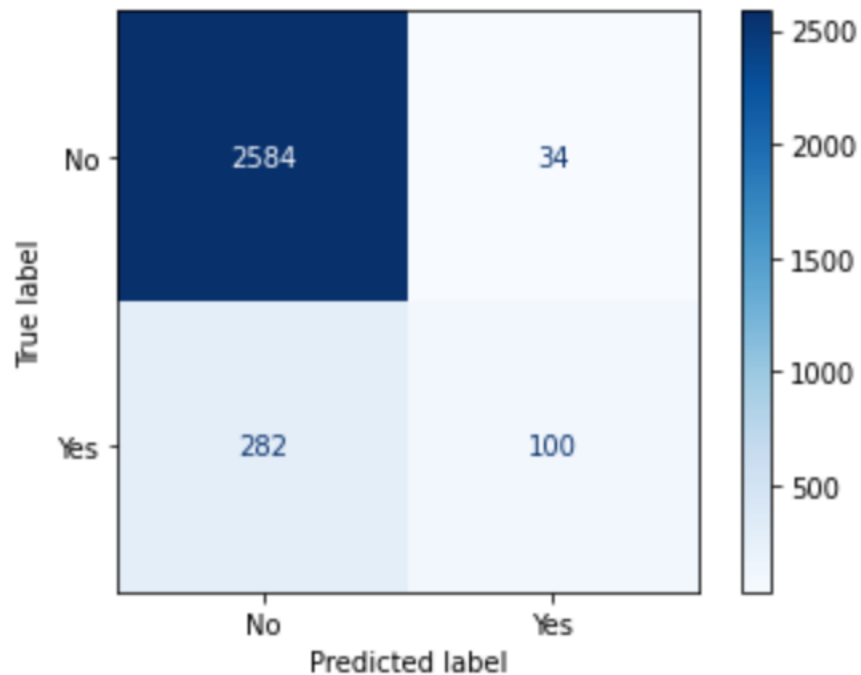
- Precision (**No**) = $TP / (TP + FP) = 2609 / (2609 + 297)$
- Precision (**Yes**) = $TN / (TN + FN) = 85 / (85 + 9)$
- Recall (**No**) = $TP / (TP + FN) = 2609 / (2609 + 9)$
- Recall (**Yes**) = $TN / (TN + FP) = 85 / (85 + 297)$
- F-measure (**a**) = $(2 \times \text{Precision}(\mathbf{a}) \times \text{Recall}(\mathbf{a})) / (\text{Precision}(\mathbf{a}) + \text{Recall}(\mathbf{a}))$
- F-measure (**b**) = $(2 \times \text{Precision}(\mathbf{b}) \times \text{Recall}(\mathbf{b})) / (\text{Precision}(\mathbf{b}) + \text{Recall}(\mathbf{b}))$



	precision	recall	f1-score	support
No	0.90	1.00	0.94	2618
Yes	0.90	0.22	0.36	382
accuracy			0.90	3000
macro avg	0.90	0.61	0.65	3000
weighted avg	0.90	0.90	0.87	3000

Step 4: Evaluate Neural Network Model Performance

Performance on the **testing data**:



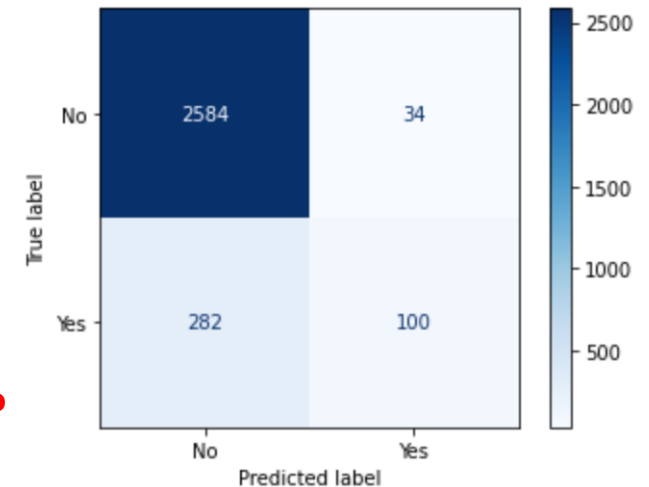
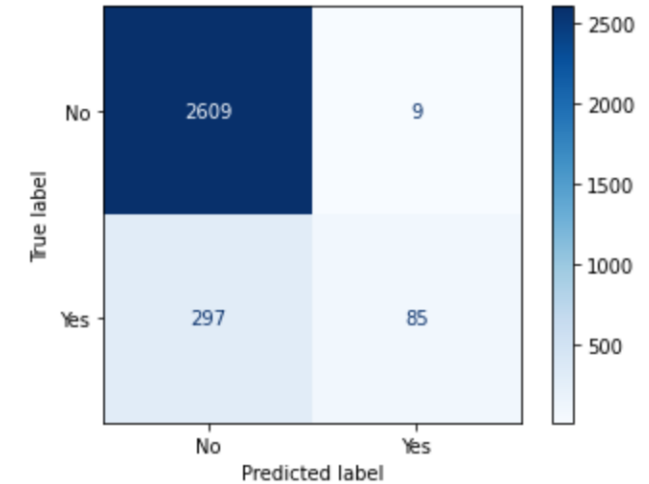
	precision	recall	f1-score	support
No	0.90	0.99	0.94	2618
Yes	0.75	0.26	0.39	382
accuracy			0.89	3000
macro avg	0.82	0.62	0.66	3000
weighted avg	0.88	0.89	0.87	3000

1. How many cars are predicted as Not bad buy?
2. How many bad buy cars are predicted as Not bad buy?
3. How many bad buy cars are identified correctly by the naive bayes model?
4. If the naive bayes predicts a car as bad buy, what is the probability that such prediction is correct?
5. Does the naive bayes model have better performance on majority (IsBadBuy = 'No') or minority class (IsBadBuy = 'Yes')?

Step 4: Evaluate Neural Network Model Performance: Model Comparison

	precision	recall	f1-score	support
No	0.90	1.00	0.94	2618
Yes	0.90	0.22	0.36	382
accuracy			0.90	3000
macro avg	0.90	0.61	0.65	3000
weighted avg	0.90	0.90	0.87	3000

	precision	recall	f1-score	support
No	0.90	0.99	0.94	2618
Yes	0.75	0.26	0.39	382
accuracy			0.89	3000
macro avg	0.82	0.62	0.66	3000
weighted avg	0.88	0.89	0.87	3000



1. Compare the performance on the majority and minority class
2. Which model is better for identifying BAD BUY cars on testing data
3. Which model has higher confidence in predicting BAD BUY cars on testing data?
4. Which model has better performance on minority class (IsBadBuy = 'Yes')? why?
5. If you are a car dealership, which model you will use, and why?

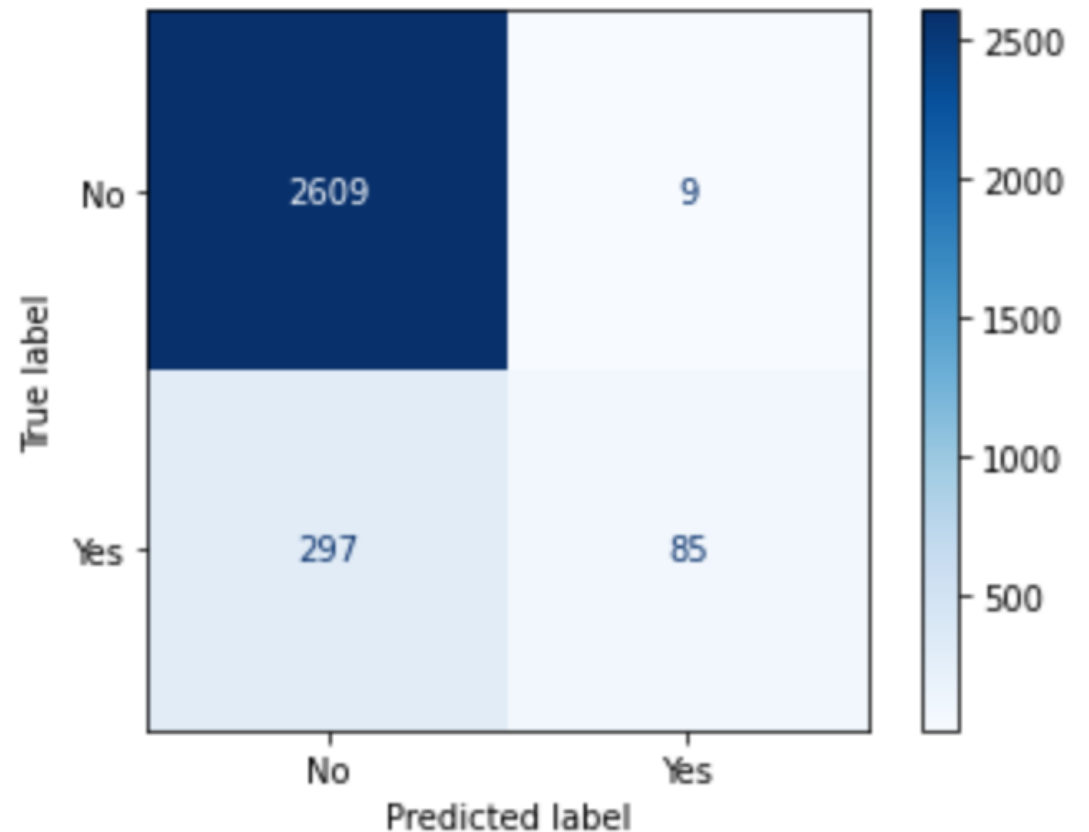
Step 4: Evaluate Neural Network Model Performance: Cost-benefit analysis

Assume the following costs / benefits:

- Cost of buying a bad car: \$2000
- Profit of purchasing a good car: \$300
- Opportunity cost of a good car that is missed: \$300

Decision Tree model:

- Cost: $297 * 2000$
- Profit: $2609 * 300$
- Opportunity cost: $9 * 300$
- Profit – Cost - Opportunity cost: 186,000



Machine Learning Basics

– Numeric Prediction

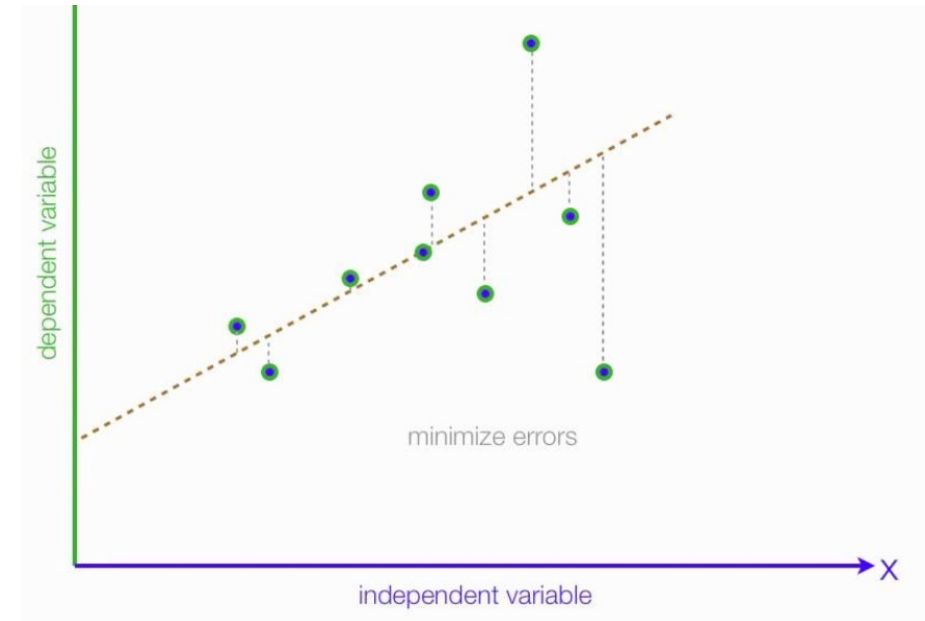
- Lecture 2&6 and lab 2&6
- Multiple choices and True False questions
- Code writing

What is Numeric Prediction

- **Numeric Prediction**

- A numeric prediction problem is when the output variable is a real value, such as “dollars” or “weight”.

- If a company’s sales have increased steadily every month for the past few years, by conducting a linear analysis on the sales data with monthly sales, the company could forecast sales in future months.



Steps to Build A Multi-Layer Neural Network

- Step 1: Medical Insurance Data
- Step 2: Data Preprocessing
- Step 3: Build a Neural Network on Training Data
- Step 4: Evaluate Neural Network Model Performance

Evaluation Metrics for Numeric Prediction

■ Mean Absolute Error (MAE)

- The **mean absolute error (MAE)** is a quantity used to measure how close forecasts or predictions are to the eventual outcomes.
- This is known as a scale-dependent accuracy measure and therefore cannot be used to make comparisons between series using different scales

$$\text{MAE} = \frac{1}{n} \sum_{t=1}^n |y_t - \hat{y}_t|$$

■ Root Mean Squared Error (RMSE)

- **RMSE** gives a relatively high weight to large errors. This means the RMSE should be more useful when large errors are particularly undesirable.

$$\text{RMSE} = \sqrt{\frac{\sum_{t=1}^n (y_t - \hat{y}_t)^2}{n}}$$

MAE: 4011.3808850910846
RMSE: 5773.7111375700415

Numeric Prediction vs Classification

- **Differences between Classification and Numeric Prediction**
 - **Target variable**
 - **Evaluation metrics**

